

company-management — idei de endpoint-uri

Ce mai poți construi pe modelul pe care îl ai deja · Rareș · 3 septembrie 2026

Înainte de orice de aici: închide întâi  -urile din `CODE_REVIEW.md` runda 4. Fiecare endpoint nou moștenește problemele celor vechi — mai ales lipsa lui `@RestControllerAdvice`, care face ca orice endpoint nou să răspundă 500 în loc de 404.

Reguli care se aplică la toate cele de mai jos:

- id inexistent → **404**, nu 500. Deci: excepție proprie + handler în advice.
- creare reușită → **201 Created** + header `Location`. Ștergere → **204 No Content** sau 200 cu corpul șters.
- colecție goală → **200 []**, niciodată excepție.
- orice câmp care intră din exterior are validare pe DTO, inclusiv `@PathVariable` și `@RequestParam`.
- un `@Modifying` bulk **nu** trece prin Bean Validation — validează tu, înainte.

Nivelul 1 — pe fix ce știi deja

derived queries + ce ai scris până acum

1. `GET /api/departments/{id}` — un singur departament

ușor

Ai listarea tuturor, dar n-ai cum să ceri unul singur. E endpoint-ul care lipsește ca să poți da `Location` după un POST.

```
GET /api/departments/3
{"id":3,"name":"Finance","location":"Iasi","budget":180000.0,"employees":[...]}
```

Gata când: id existent → 200 cu departamentul · id inexistent → 404 cu mesaj, nu 500 · angajații vin fără interogare suplimentară.

Ai deja `left join fetch` pentru listă. Aceași idee, cu `where d.id = :id`.

2. `GET /api/employees/{id}` — un singur angajat

ușor

Perechea celui de sus, pe cealaltă entitate. Îl vei folosi în teste ca să verifici ce a scris un PUT sau un PATCH — chiar acum n-ai cum să citești salariul unui angajat.

Gata când: răspunsul conține salariul și numele departamentului · 404 pe id inexistent.

3. `GET /api/departments/{id}/employees` — angajații unui departament

ușor

Aceași informație pe care o ai deja în listarea de departamente, dar adresabilă separat — clientul nu mai trebuie să descarce toate departamentele ca să vadă echipa unuia.

Gata când: departament fără angajați → 200 `[]` · departament inexistent → 404 (atenție: cele două cazuri sunt diferite, nu le confunda).

Distincția din DoD e tot subiectul: „colecție goală” ≠ „resursa părinte nu există”.

4. GET /api/employees?jobTitle=Java Developer — filtrare după funcție

ușor

Primul tău parametru de query. Când lipsește, endpoint-ul se comportă exact ca acum (întoarce tot).

```
GET /api/employees?jobTitle=QA Engineer
[{"id":3,"firstName":"Maria","lastName":"Ionescu","departmentName":"IT"}]
```

Gata când: fără parametru → toți angajații · cu parametru → doar cei care se potrivesc · valoare fără rezultate → 200 [].

Spring Data îți construiește interogarea din numele metodei: `findByJobTitle(String jobTitle)`. Nu scrii SQL. `@RequestParam(required = false)`.

Nivelul 2 — noțiuni noi, dar vecine

interogări derivate compuse + relația în ambele sensuri

5. GET /api/employees?minSalary=7000&maxSalary=10000 — interval de salariu

mai greu

Doi parametri opționali care lucrează împreună. Cazul interesant e când vine doar unul dintre ei.

Gata când: ambii → interval închis · doar `minSalary` → toți peste · doar `maxSalary` → toți sub · niciunul → toți · `minSalary > maxSalary` → 400 cu mesaj, nu listă goală.

`findBySalaryBetween`, `findBySalaryGreaterThanEqual`, `findBySalaryLessThanEqual`. Ultima condiție din DoD nu se rezolvă cu o adnotare — e o verificare pe care o scrii tu.

6. PATCH /api/employees/{id}/department — mută angajatul în alt departament

mai greu

Primul endpoint care atinge **relația**, nu doar câmpurile. Corpul cererii: `{"departmentId": 2}`.

```
PATCH /api/employees/1/department {"departmentId":2}
{"id":1,"firstName":"Rares","lastName":"Dumitru","departmentName":"HR"}
```

Gata când: angajatul apare în lista noului departament **și** a dispărut din a vechiului, verificat prin `GET /api/departments/{id}/employees` pe amândouă · departament inexistent → 404, iar angajatul rămâne unde era.

Ai deja `addEmployee` și `removeEmployee` pe `Department`. Sunt exact uneltele pentru asta — folosește-le pe amândouă, în ordinea corectă. Întrebarea de pus: care capăt al relației e cel pe care baza de date îl citește când scrie `department_id`?

7. GET /api/departments/{id}/payroll — cât costă lunar un departament

mai greu

Prima ta agregare. Suma salariilor angajaților dintr-un departament, calculată în baza de date, nu în Java.

```
GET /api/departments/1/payroll
{"departmentId":1,"departmentName":"IT","employeeCount":3,"totalSalary":23800.00}
```

Gata când: rezultatul vine dintr-un singur SELECT (verifici cu `show-sql`) · departament fără angajați → `employeeCount: 0` și `totalSalary: 0`, nu `null` · departament inexistent → 404.

`@Query("select sum(e.salary) from Employee e where e.department.id = :id")`. Cazul „fără angajați” e capcana: în SQL, `sum` pe zero rânduri dă `NULL`, nu 0.

8. GET /api/employees/search?q=pop — căutare după nume, parțială

mai greu

Caută în prenume **sau** nume, fără să conteze literele mari. „pop” găsește și „Popescu”, și „Popa”.

Gata când: „POP”, „pop” și „Pop” dau același rezultat · `q` gol sau lipsă → 400 cu mesaj (o căutare fără termen nu e o căutare) · niciun rezultat → 200 `[]`.

`findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCase(...)`. Da, e lung — ăsta e semnalul că ai ajuns la limita interogărilor derivate și că un `@Query` ar fi mai lizibil. Scrie-le pe amândouă și compară.

Nivelul 3 — provocări

paginare, proiecții, tranzacții

9. GET /api/employees?page=0&size=5&sort=salary,desc — paginare și sortare

provocare

Astăzi, dacă ai 10.000 de angajați, `GET /api/employees` îi trimite pe toți. Paginarea e răspunsul, și Spring Data ți-o dă aproape gratis.

```
GET /api/employees?page=0&size=2&sort=salary,desc
{"content":[{"...},{...}], "totalElements":7, "totalPages":4, "number":0, "size":2}
```

Gata când: repository-ul primește `Pageable` și întoarce `Page<Employee>` · nu construiești tu `page / size` din `@RequestParam` · `sort=salary,desc` chiar sortează · pagină peste ultima → 200 cu `content: []`.

Semnătura e `findAll(Pageable pageable)`, iar în controller pui direct `Pageable pageable` ca parametru — Spring îl construiește singur din query string. Partea de gândit: `Page<Employee>` conține entități, iar tu trebuie să trimiți DTO-uri. Caută `Page.map`.

10. GET /api/departments/stats — un rând pe departament, cu agregări

provocare

Nume, număr de angajați, salariu mediu, salariu maxim — pentru toate departamentele, într-o singură interogare.

```
GET /api/departments/stats
[{"name":"IT","employeeCount":3,"averageSalary":7933.33,"maxSalary":9500.00},
 {"name":"HR","employeeCount":2,"averageSalary":6600.00,"maxSalary":7200.00}]
```

Gata când: un singur SELECT pentru tot răspunsul (verificat cu `show-sql`) · departamentele fără angajați apar și ele, cu 0 · nu încarci nicio entitate Employee în memorie.

Două drumuri: proiecție pe interfață (declari o interfață cu `getName()`, `getEmployeeCount()` ... și Spring o umple singur) sau `select new ...DepartmentStats(d.name, count(e), avg(e.salary))`. Ultima condiție din DoD îți spune ce fel de `join` îți trebuie ca departamentele goale să nu dispară din rezultat.

11. POST /api/departments/{id}/employees/bulk — angajări în lot, tot sau nimic

provocare

Primești o listă de angajați și îi creezi pe toți într-un departament. Dacă **unul singur** e invalid sau are un email duplicat, nu se salvează **niciunul**.

```
POST /api/departments/2/employees/bulk
[ {...ok...}, {...email duplicat...} ]
-> 409, iar GET /api/departments/2/employees arata exact ce era inainte
```

Gata când: lot valid → 201 cu toți angajații creați · lot cu o singură eroare → 4xx **și zero rânduri noi în baza de date**, verificat prin GET după · validarea prinde elementele listei, nu doar lista.

Ultima condiție e cea care te trimite la ceva ce ai mai văzut o dată: `@Valid` pe un `@RequestBody List<...>` nu coboară singur în elemente. Iar „tot sau nimic” e definiția unei tranzacții — ai deja `@Transactional` pe servicii; întrebarea e ce anume declanșează rollback-ul.

12. GET /api/departments/{id}/employees/top?limit=3 — cei mai bine plătiți

provocare

Primii N angajați dintr-un departament, ordonați descrescător după salariu. Limita vine din URL și trebuie să fie rezonabilă.

Gata când: limitarea se face **în baza de date** (SQL-ul conține `limit / fetch first`), nu cu `.stream().limit()` în Java · `limit` lipsă → 3 implicit · `limit=0` sau negativ → 400 · departament cu 2 angajați și `limit=10` → 2 rezultate, nu eroare.

Prima condiție e miezul: dacă filtrezi în Java, ai adus deja toate rândurile din baza de date — exact lucrul pe care încerci să-l eviți. `PageRequest.of(0, limit, Sort.by("salary").descending())`.

la-le pe rând, de sus în jos. Câte un commit per endpoint, cu buildul rulat înainte — și `grep` după fiecare regulă închisă.